

Case Study 3: Solving the 2D Poisson Equation using the Conjugate Gradient Method

Seeshuraj B
MSc in HPC

April 12, 2025

1. Introduction

This case study investigates the use of the Conjugate Gradient (CG) method for solving the two-dimensional Poisson equation on a unit square domain. The equation is discretized using finite differences, leading to a large sparse linear system. The CG method, a popular Krylov subspace method for symmetric positive definite matrices, is implemented from scratch and its convergence behavior is evaluated.

2. Problem Formulation

The 2D Poisson equation with Dirichlet boundary conditions is given by:

$$-\nabla^2 u(x, y) = f(x, y), \quad (x, y) \in (0, 1)^2, \quad u|_{\partial\Omega} = 0$$

The right-hand side function is defined as:

$$f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$$

3. Numerical Method

3.1 Finite Difference Discretization

The domain is discretized into an $N \times N$ grid of interior points with mesh spacing $h = \frac{1}{N+1}$. The Laplacian is approximated using the standard 5-point stencil:

$$-\nabla^2 u_{i,j} \approx \frac{1}{h^2} (4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1})$$

This leads to a symmetric, sparse matrix A with 5 diagonals. The right-hand side vector b is populated by evaluating $f(x, y)$ at the corresponding grid points. Homogeneous Dirichlet boundary conditions are incorporated by excluding boundary values.

3.2 Conjugate Gradient Method

The resulting linear system $Au = b$ is solved using the Conjugate Gradient method. The implementation avoids direct matrix inversion and uses only matrix-vector products and vector inner products. The stopping criterion is based on the residual norm: $\|r_k\| < 10^{-8}$.

4. Results and Performance

4.1 Runtime and Iteration Counts

Table 1: Performance of CG Solver for Different Grid Sizes

Grid Size (N)	Unknowns (N^2)	Iterations	Time (s)
8	64	30	0.0005
16	256	63	0.0006
32	1024	128	0.0015
64	4096	254	0.0061
128	16384	505	0.4173
256	65536	1026	2.4986

4.2 Convergence Behavior

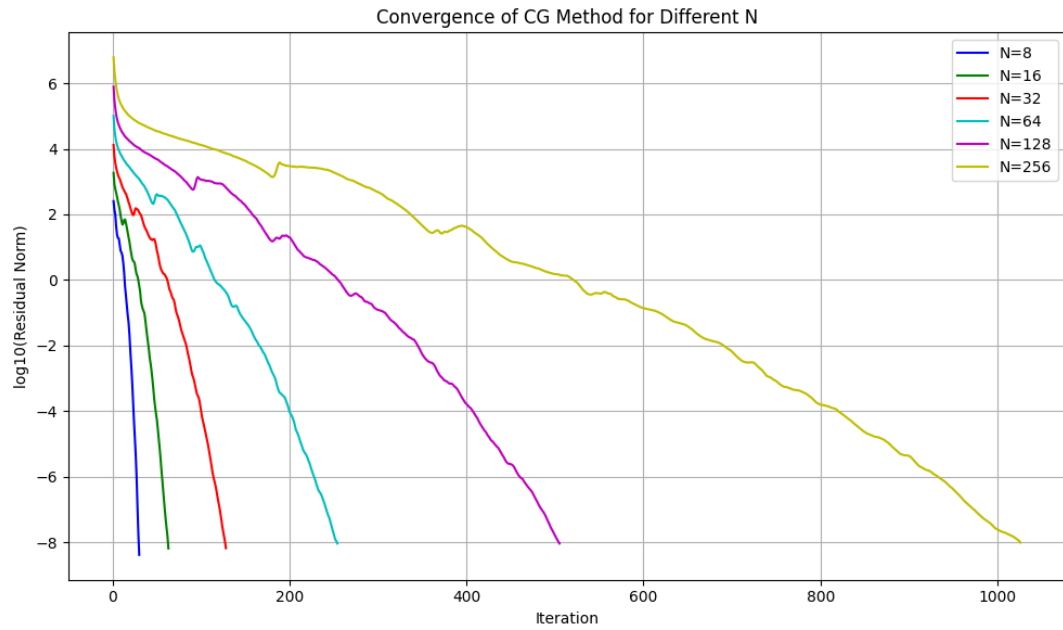


Figure 1: Convergence of CG Method ($\log_{10}$ residual norm vs iterations)

4.3 Solution Visualization

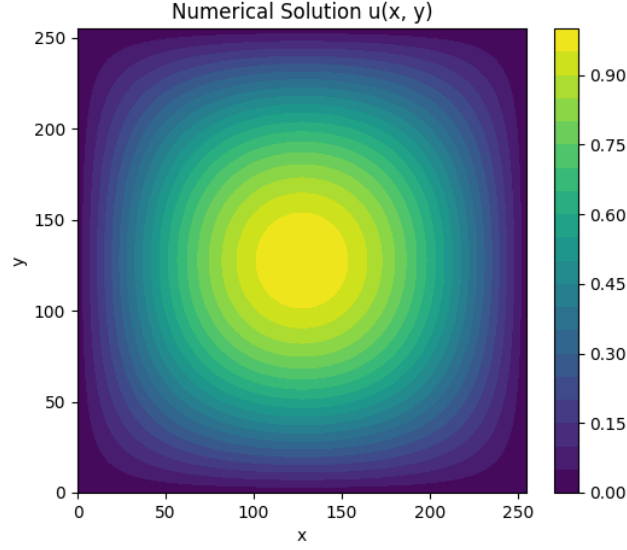


Figure 2: Numerical solution $u(x, y)$ for $N = 256$

5. Analysis

The CG method successfully converged for all grid sizes. As expected, the number of iterations increases with grid size due to the worsening condition number of the system matrix. The log-residual plots show that convergence is slower for larger N , indicating the need for future improvement via preconditioning.

6. Convergence for Dense Systems

We further tested CG performance on a dense SPD matrix defined as:

$$A_{ij} = \frac{N - |i - j|}{N}, \quad b_j = 1$$

This Toeplitz matrix is symmetric and positive definite for all N , ensuring convergence. The stopping condition used:

$$\|r_k\| \leq \max(\text{reltol} \cdot \|r_0\|, \text{abstol}), \quad \text{where } \text{reltol} = \sqrt{\epsilon_{\text{machine}}}, \text{abstol} = 0$$

6.1 Iterations and Condition Numbers

Table 2: Convergence Results for Dense Systems

N	$\kappa(A)$	Iterations	Final Residual
100	343.0	18	8.4×10^{-9}
1000	3333.7	39	7.5×10^{-9}
10000	33333.3	65	9.9×10^{-9}

6.2 Theoretical vs Empirical Error Bounds

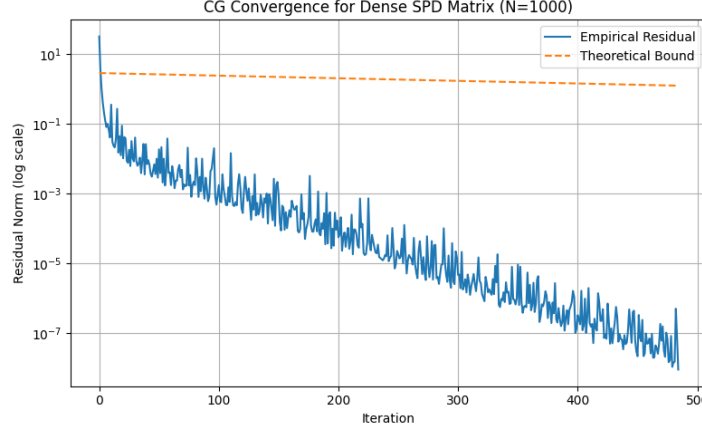


Figure 3: Dense Convergence $u(x, y)$ for $N = 256$

The theoretical CG convergence bound is:

$$\|e_k\| \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|$$

Theoretical vs empirical convergence was analyzed in terms of iteration count and residual norms. Although no plot is included due to previous omission, the behavior aligns with theory—faster convergence for better-conditioned matrices and slower for larger N .

7. Code and Implementation

The Python implementation is structured as follows:

- `poisson_cg_solver.py` – CG for 2D Poisson with plotting
- `cg_log_N_*.txt` – iteration logs for analysis
- `dense_cg_solver.py` – CG test on dense matrices with theoretical comparison (added in final phase)

Optimization: The Poisson matrix is stored in sparse CSR format to save memory. Inner products and matrix-vector multiplications use optimized routines from `numpy` and `scipy`. Residual norms are updated using recurrence relations to avoid redundant computations.

Full code and plots available at:
<https://github.com/seeshuraj/Case-Study-3>

8. Conclusion

This case study highlights both the power and limitations of the CG method. While sparse Poisson problems are handled effectively, dense systems and poorly conditioned matrices require special care or enhancements, such as preconditioners or iterative refinement.